ACTIVIDAD 3 · CIBERSEGURIDAD

Seguridad en la nube, contenedores y APIs

Laboratorio de infraestructura como código, análisis de imágenes y autenticación JWT con verificación de seguridad.

Terraform · LocalStack · Docker · Trivy · JWT · OWASP ZAP



Alumno: Hugo Ruiz

Materia: Ciberseguridad

Fecha: noviembre de 2026

Resumen visual

Fase 1 · S3 seguro

3 recursos Terraform · AES-256 · 4/4 bloqueos públicos · ACL privada

Fase 2 · Contenedores

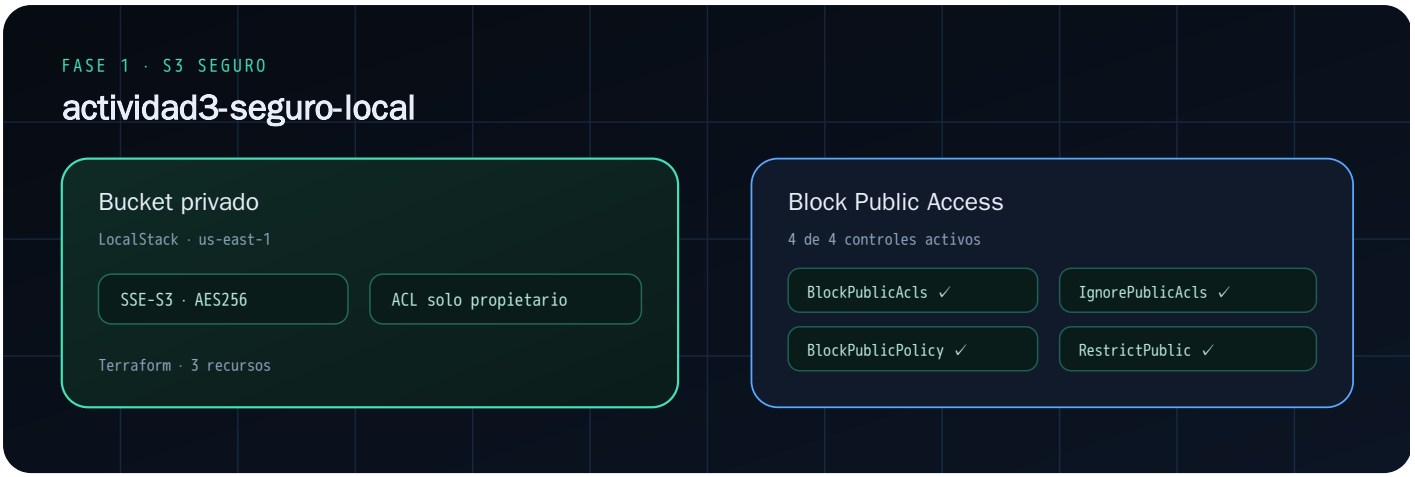
595 hallazgos y 23 críticas en `node:14` →
0 / 0
en `node:18-alpine`

Fase 3 · API + sitio

JWT HS256 · 401 sin token · 403 sin rol · 200 con admin · ZAP 0 FAIL

Fase	Tema	Herramientas	Resultado clave
1	Infraestructura como código	Terraform · LocalStack · AWS CLI	Bucket privado, AES-256, 4/4 bloqueos
2	Seguridad de contenedores	Docker · Trivy	595 hallazgos → 0
3	Desarrollo seguro y autenticación	Node.js · Express · JWT · OWASP ZAP	401 / 403 / 200 · 0 FAIL

Fase 1 · Bucket S3 seguro



Recursos declarados

Recurso	Configuración
`aws_s3_bucket`	`actividad3-seguro-local`
`aws_s3_bucket_server_side_encryption_configuration`	`SSEAlgorithm = AES256`
`aws_s3_bucket_public_access_block`	4 controles en `true`

Verificación real

Consulta	Resultado
`terraform apply`	3 added, 0 changed, 0 destroyed
`get-bucket-encryption`	"SSEAlgorithm": "AES256"
`get-public-access-block`	4/4 en true
`get-bucket-acl`	Solo propietario con FULL_CONTROL
`get-bucket-policy-status`	NoSuchBucketPolicy
Segundo `terraform plan`	No changes

> Lectura importante: `NoSuchBucketPolicy` solo indica que no hay una política de bucket adjunta. No significa que el bucket sea público. La privacidad se sostiene con la ACL privada y los cuatro bloqueos.

Acceso anónimo en LocalStack

```
HTTP/1.1 200 OK
x-amz-server-side-encryption: AES256
```

LocalStack permitió la lectura anónima. Es una limitación del simulador: no replica todas las reglas de autorización de AWS. La respuesta confirma que el objeto está cifrado con AES-256. En AWS real, con los cuatro bloqueos activos y sin política pública, la lectura anónima se deniega.

Fase 2 · Seguridad de contenedores



Imagen	SO	Node/npm	Total	Críticas	Altas
`act3-app:node14`	566	29	595	23	572
`act3-app:node18`	0	0	0	0	0

Mitigaciones aplicadas

Medida	Efecto
`node:14` → `node:18-alpine`	Base más reciente y menor superficie
`apk upgrade --no-cache`	Corrige OpenSSL y zlib del sistema
Build multi-stage	Solo pasan a la imagen final las dependencias de producción
Eliminar `npm`, `npx` y `corepack`	Quita del runtime paquetes vulnerables que la app no necesita
`USER node`	La aplicación no corre como root

Dockerfile final

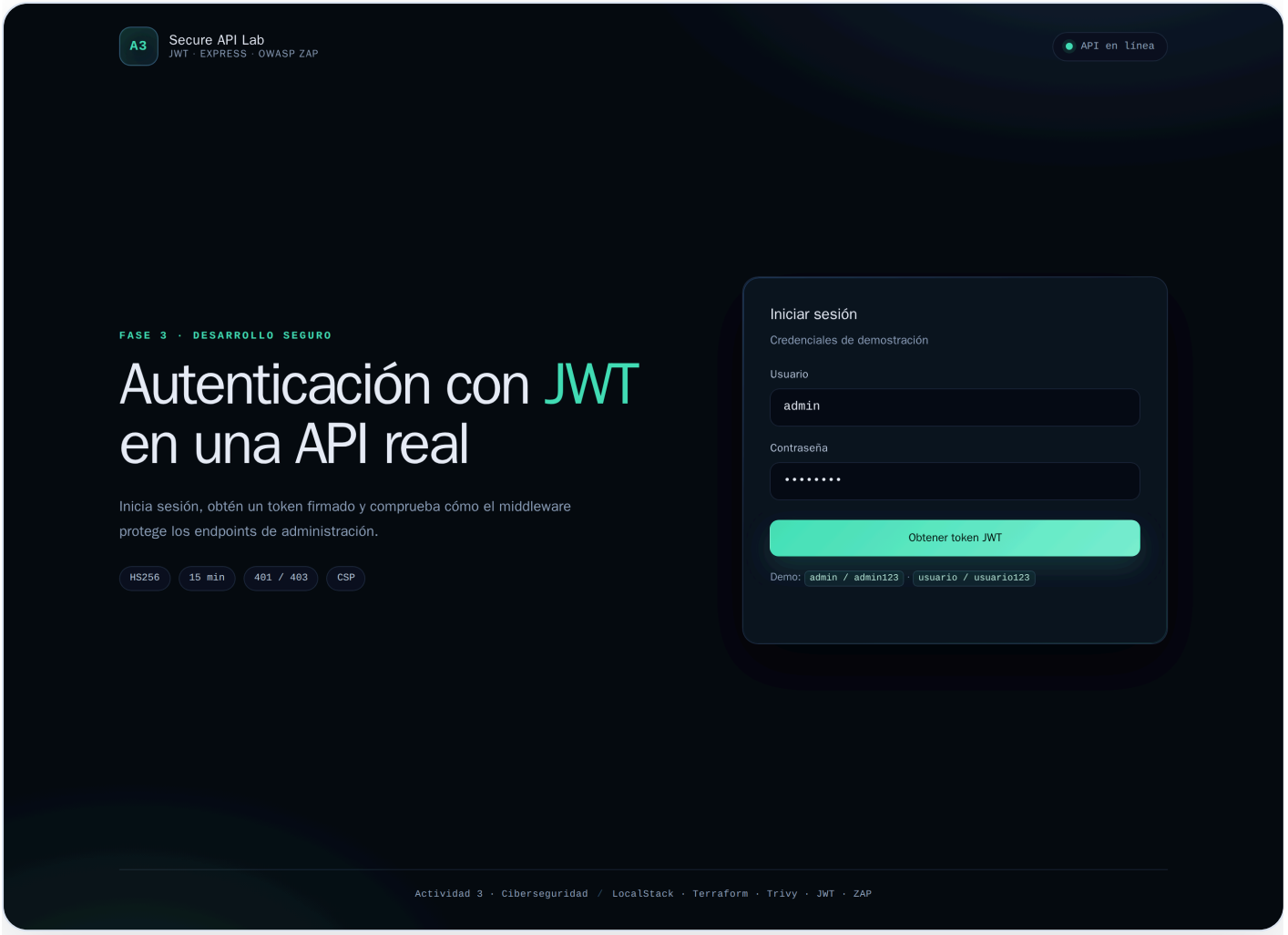
```
FROM node:18-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:18-alpine
RUN apk upgrade --no-cache \
    && rm -rf /usr/local/lib/node_modules/npm \
              /usr/local/lib/node_modules/corepack \
              /usr/local/bin/npm /usr/local/bin/npx /usr/local/bin/corepack
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json server.js ./
```

```
ENV NODE_ENV=production
USER node
CMD ["node", "server.js"]
```

> La actividad pide `node:18`. A noviembre de 2026 Node 18 ya está fuera de soporte; para un proyecto real se recomienda `node:22-alpine` o superior.

Fase 3 · Sitio y API con JWT



A3 Secure API Lab
JWT · EXPRESS · OWASP ZAP

API en linea

FASE 3 · DESARROLLO SEGURO

Autenticación con JWT en una API real

Inicia sesión, obtén un token firmado y comprueba cómo el middleware protege los endpoints de administración.

HS256 | 15 min | 401 / 403 | CSP

Iniciar sesión

Credenciales de demostración

Usuario

admin

Contraseña

••••••••

Obtener token JWT

Demo: `admin / admin123` · `usuario / usuario123`

Token obtenido correctamente.

SESIÓN ACTIVA

Panel de administración

Cerrar sesión

Token JWT

role: admin | exp: 11:51:27 PM


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiIsImlhdCI6MTc5NDI2NzM4NywiZXhwIjoxNzk0MjY4Mjg3LCJhdWQiOiJhY3QzLXdlYiIsImlzcyI6ImFjdDMtYXBpIn0.-8np7ZVwArUSkGFsuvBgDAfEKBIHqmSrKQj0rplIgTA
```


Se guarda en `sessionStorage` y se envía como `Authorization: Bearer <token>`.

Pruebas de autorización

Compara el código HTTP con y sin token.

GET /api/public | GET /api/me | GET /api/admin

GET /api/admin sin token

Ejecuta una prueba para ver el código HTTP y la respuesta.

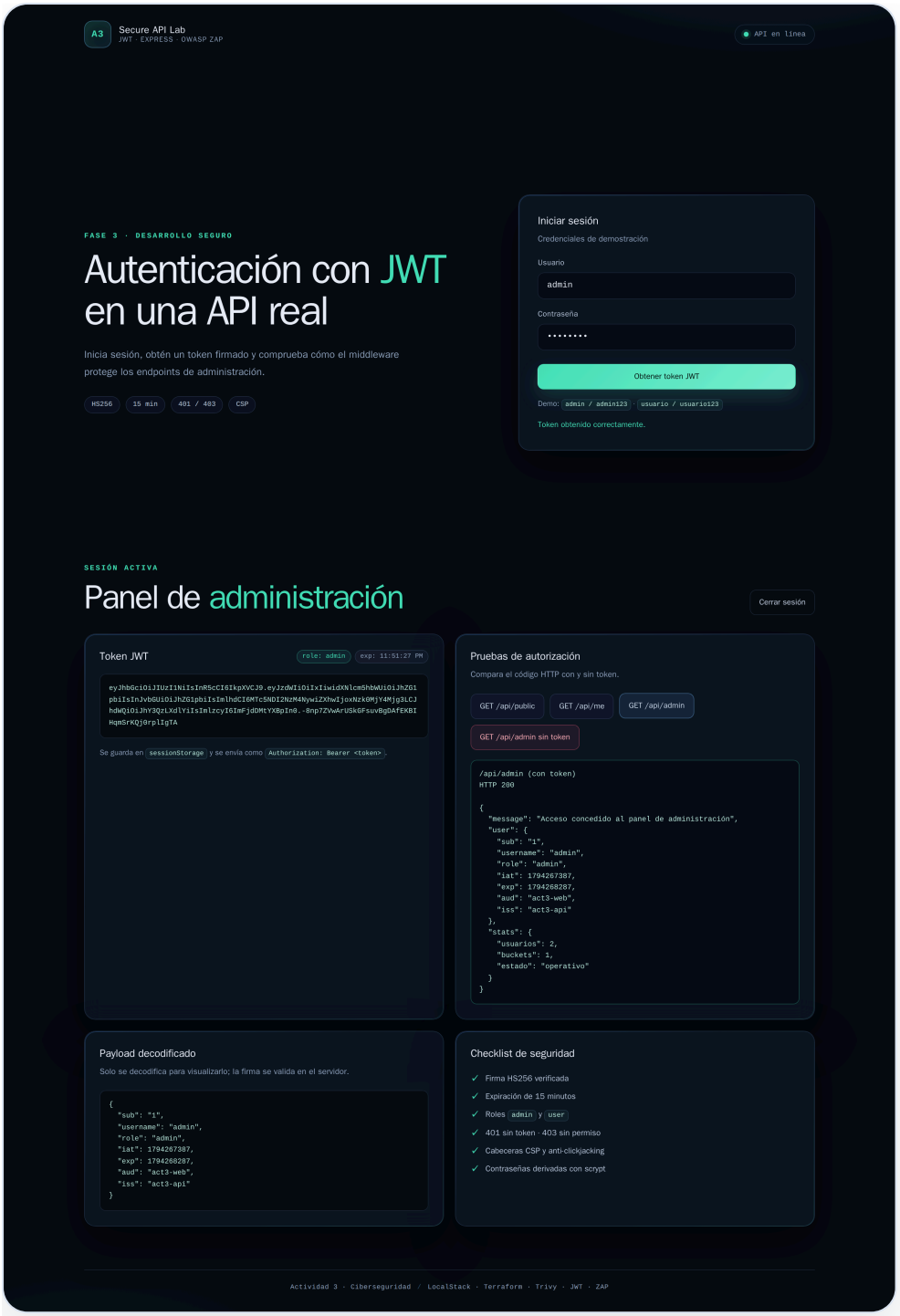
Payload decodificado

Solo se decodifica para visualizarlo; la firma se valida en el servidor.


```
{
  "sub": "1",
  "username": "admin",
  "role": "admin",
  "iat": 1794267387,
  "exp": 1794268287,
  "aud": "act3-web",
  "iss": "act3-api"
}
```


Checklist de seguridad

- ✓ Firma HS256 verificada
- ✓ Expiración de 15 minutos
- ✓ Roles `admin` y `user`
- ✓ 401 sin token · 403 sin permiso
- ✓ Cabeceras CSP y anti-clickjacking
- ✓ Contraseñas derivadas con scrypt



Matriz de autorización

Prueba	Código	Resultado
`GET /api/admin` sin token	401	Correcto
`GET /api/admin` con token inválido	401	Correcto
`GET /api/admin` con token de `usuario`	403	Correcto
`GET /api/admin` con token de `admin`	200	Correcto

Middleware JWT

```
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token requerido' });
  }

  const token = authHeader.slice(7).trim();

  try {
    req.user = jwt.verify(token, JWT_SECRET, {
      algorithms: ['HS256'],
      issuer: 'act3-api',
      audience: 'act3-web'
    });
    return next();
  } catch (error) {
    return res.status(401).json({ error: 'Token inválido o expirado' });
  }
}

function requireRole(role) {
  return (req, res, next) => {
    if (!req.user || req.user.role !== role) {
      return res.status(403).json({ error: 'Acceso denegado' });
    }
    return next();
  };
}
```

OWASP ZAP

Métrica	Valor
FAIL	0
WARN	3
PASS	65

Las 3 advertencias son de bajo riesgo: contenido no almacenable, formulario de login detectado (esperado) y ausencia de token anti-CSRF. La API usa JSON + `Authorization: Bearer`, por lo que el riesgo CSRF es bajo. Se documenta como mejora.

Política IAM de mínimo privilegio

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeerObjetosDelBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::actividad3-seguro-local/*"
    },
    {
      "Sid": "ListarSoloEseBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::actividad3-seguro-local"
    }
```

```
  ]
}
```

Conclusiones

Aprendizaje	Aplicación
IaC permite repetir y auditar la infraestructura	Terraform declara bucket, cifrado y bloqueos
La configuración declarada no siempre es la efectiva	Se verificó con AWS CLI y se documentó la limitación de LocalStack
Las imágenes obsoletas acumulan vulnerabilidades	`node:14` tenía 595 hallazgos; la imagen corregida, 0
Autenticación no es lo mismo que autorización	`401` sin token, `403` con token sin rol
El mínimo privilegio reduce el impacto	IAM solo permite `s3:GetObject` y `s3:ListBucket`
Las pruebas automatizadas complementan el análisis	ZAP: 0 FAIL; pruebas manuales: 401/403/200

Evidencias y anexos

Anexo	Ubicación
Scripts Terraform	`fase1/`
Reportes Trivy	`fase2/evidencias/`
Código de la API	`fase3/server.js`
Política IAM	`fase3/iam-policy.json`
Diagrama editable	`docs/arquitectura.drawio`
Capturas del sitio	`docs/capturas/`

Actividad 3 · Ciberseguridad · Terraform · LocalStack · Docker · Trivy · JWT · OWASP ZAP